Shell Scripting

1. Introduction to Shell Scripting

Shell scripting automates repetitive Linux/Unix tasks using command sequences in a script file (usually ending with .sh).
Example:

```
#!/bin/bash

echo "Hello, World!"
```

Here, #!/bin/bash specifies the Bash shell.

2. What is a Shell?

A shell is an interface between the user and the operating system kernel that executes commands.

3. Different Types of Shells

Shell	Description	Path
Bourne Shell (sh)	Original Unix shell	/bin/sh
Bash	Bourne Again Shell, most common	/bin/bash
C Shell (csh)	C-like syntax	/bin/csh
Korn Shell (ksh)	Combines sh and csh features	/bin/ksh
Z Shell (zsh)	Advanced shell	/bin/zsh

4. Variables in Shell Scripting

```
name="Deva"

echo "My name is $name"

read age

echo "You are $age years old."
```

5. Conditionals

```
num=10

if [ $num -gt 5 ]; then
```

```
echo "Number is greater than 5"
else
echo "Number is less than or equal to 5"
fi
```

6. Loops

For Loop:

```
for i in 1 2 3 4 5; do
echo "Number: $i"
done
```

While Loop:

```
count=1
while [ $count -le 5 ]; do
echo $count
((count++))
done
```

7. Functions

```
greet() {
  echo "Hello, $1!"
}

greet Deva
```

8. Passing Values to Functions

```
add() {
  sum=$(($1 + $2))
  echo "Sum is $sum"
}
add 10 20
```

9. Basic Operators

Type	Example	Description
Arithmetic	$((a + b))	Addition
Assignment	a=10	Assign value
Logical	[$a -gt 5 -a $b -lt 10]	AND condition
Boolean	! false	NOT operator
String	["$str1" = "$str2"]	Compare strings

10. String Operators

Operator	Example	Description
=	["$a" = "$b"]	Equal strings
!=	["$a" != "$b"]	Not equal
-z	[-z "$a"]	Empty string
-n	[-n "$a"]	Non-empty string

11. Relational Operators

Operator	Meaning
-eq	Equal to
-ne	Not equal to
-gt	Greater than
-lt	Less than
-ge	Greater than or equal to
-le	Less than or equal to

Example:

```
a=10; b=20
if [ $a -lt $b ]; then
```

```
echo "$a is less than $b"
fi
```

12. File Test Operators

Operator	Description
-e file	File exists
-f file	Regular file
-d file	Directory
-r file	Readable
-w file	Writable
-x file	Executable
-s file	File not empty
-L file	Symbolic link
!	NOT operator

Example:

```
file="test.txt"
if [ -e $file ]; then
echo "$file exists"
else
echo "$file does not exist"
fi
```

13. Positional Parameters

Positional parameters represent values passed to a script.

Parameter	Description
$0	Script name
$1, $2, ...	First, second, etc., arguments
$#	Number of arguments

Parameter	Description
$@	All arguments (separate)
$*	All arguments (as one string)
$?	Exit status of last command
$$	Script process ID

Example:

```
#!/bin/bash
echo "Script Name: $0"
echo "First Arg: $1"
echo "Second Arg: $2"
echo "Total Args: $#"
```

Run: bash script.sh Deva 21
Output:

Script Name: script.sh

First Arg: Deva

Second Arg: 21

Total Args: 2

14. Passing Arguments to a Script

Arguments are passed when executing the script.
Example:

```
#!/bin/bash
name=$1
age=$2
echo "Hello $name, you are $age years old."
```

Run:

bash info.sh Deva 21

Output:

Hello Deva, you are 21 years old.

✅ Summary:

Shell scripting automates Linux tasks using variables, loops, conditionals, functions, and arguments. Operators help perform calculations and comparisons, while file and positional parameters enhance script control and flexibility.